

GCT - Training Program

5-6 Week University Full-Stack Developer Training & Selection Program

Objective

Create a short, practical training and evaluation program for university students to identify strong candidates for Banfico.

The program will focus on full-stack development using:

- Java
- JavaScript
- Spring Boot
- Hibernate / JPA
- PostgreSQL
- Docker
- Git
- Nginx as API Gateway
- Keycloak

The objective is not to teach basics from scratch. Students should complete prerequisite self-learning / Udemy courses before attending Banfico-led sessions.

Time Commitment

- Duration: 5-6 weeks
- Training Time: 6 hours per week
- Suggested Format: 2 sessions per week, with Session 1 limited to 1.5 hours for Banfico team walkthrough

Example Weekly Split

- Session 1: 1.5 hours only - task explanation, expected output, project walkthrough, and clarification of acceptance criteria.
- Session 2: Lab review, debugging, code review, implementation feedback, and evaluation.
- Students should spend additional self-learning / practice time outside these 6 hours.

Training Scope Clarification

The 36 hours are intended for practical project guidance, implementation review, debugging support, and evaluation. The basics listed below must be completed before the program starts so the sessions can focus on hands-on development and selection.

Banfico Session 1 Clarification

- Banfico team members should keep Session 1 to 1.5 hours only.
- Session 1 should be used only to explain the weekly task, expected output, acceptance criteria, repository structure, and walkthrough of the required implementation approach.
- Concepts should not be taught in detail during Session 1. Candidates must learn the concepts themselves through prerequisite self-learning / Udemy / documentation before or alongside the weekly task.
- Banfico team involvement should focus on practical guidance, review, debugging support, and evaluation rather than classroom-style concept training.

Pre-Requisite Learning

Before the program starts, students should complete basic learning on:

- Java basics
- JavaScript basics
- HTML/CSS basics
- Git basics
- SQL basics
- REST API basics
- Docker basics
- HTTP basics
- Spring Boot basics
- Hibernate / JPA basics

Banfico team should not spend training time on these basics.

Expected Under Basics for Pre-Requisites

Students are expected to already understand and practice the following basics before joining the Banfico-led sessions.

Java Basics

- Variables, data types, operators, loops, and conditional statements.
- Methods, classes, objects, constructors, and basic OOP concepts.
- Collections such as List, Set, Map, and basic iteration.
- Exception handling using try/catch and custom error messages.
- Basic Maven or Gradle project execution.
- Ability to write and run a simple Java program independently.

Spring Boot Basics

- Understanding what Spring Boot is and why it is used for backend API development.
- Creating a basic Spring Boot project using Spring Initializr or an IDE.
- Basic project structure: controller, service, repository, DTO, entity, and configuration packages.
- Using annotations such as @SpringBootApplication, @RestController, @RequestMapping, @GetMapping, @PostMapping, @Service, and @Repository.
- Running a Spring Boot application locally and debugging common startup issues.
- Understanding application.properties / application.yml at a basic level.
- Basic request validation, exception handling, and logging concepts.

Hibernate / JPA Basics

- Understanding ORM, JPA, Hibernate, entity, repository, and database table mapping concepts.
- Using annotations such as @Entity, @Table, @Id, @GeneratedValue, @Column, and basic relationship mappings.
- Understanding basic Spring Data JPA repositories and CRUD operations.
- Mapping simple one-to-many and many-to-one relationships at a basic level.
- Understanding transactions, lazy/eager loading at a basic level, and common entity mapping errors.
- Ability to connect Spring Boot with PostgreSQL and persist/retrieve data using JPA.

JavaScript, HTML, and CSS Basics

- HTML page structure, forms, input fields, buttons, and tables.
- Basic CSS styling, layout, and form alignment.
- JavaScript variables, functions, arrays, objects, conditions, and loops.
- DOM manipulation and basic event handling.
- Calling APIs using fetch or Axios and handling JSON responses.

Git Basics

- Git clone, status, add, commit, push, pull, and branch usage.
- Creating meaningful commit messages.
- Resolving simple merge conflicts.
- Understanding repository structure and README updates.

SQL and PostgreSQL Basics

- Database, schema, table, row, column, primary key, and foreign key concepts.
- Basic SELECT, INSERT, UPDATE, and DELETE queries.
- WHERE conditions, ORDER BY, LIMIT, and simple joins.
- Ability to create simple tables and verify data using SQL client tools.

REST API and HTTP Basics

- Understanding client-server communication.
- HTTP methods: GET, POST, PUT/PATCH, and DELETE.
- Common status codes such as 200, 201, 400, 401, 403, 404, and 500.
- Request headers, request body, path parameters, and query parameters.
- JSON request and response structure.

- Testing APIs using Postman.

Docker Basics

- What Docker is and why containers are used.
- Basic Docker commands: build, run, ps, logs, stop, and rm.
- Understanding Dockerfile at a basic level.
- Basic Docker Compose usage to run multiple services locally.

Basic Developer Readiness

- Install and use IntelliJ IDEA or VS Code.
- Run a local application and debug common startup issues.
- Read logs and identify simple errors.
- Use Google, official documentation, and error messages to unblock simple issues.
- Communicate blockers clearly with screenshots, logs, and steps to reproduce.

Students should already be able to:

- Write basic Java code.
- Create and run a basic Spring Boot application.
- Use Hibernate / JPA for simple database persistence.
- Understand JSON request / response.
- Use Git clone / commit / push / pull.
- Write basic SQL queries.
- Understand HTTP methods.
- Run a simple application locally.

Program Project

The full program will be built around one practical banking-style project:

Mini Banking / Open Banking Consent Management System

The project will gradually evolve across the weeks.

Final Project Features

- Customer management
- Bank account management
- Transaction history
- Beneficiary management
- Consent creation
- Consent approval / rejection
- JWT-protected APIs
- Role-based access
- Nginx Gateway
- Keycloak authentication
- PostgreSQL persistence
- Docker Compose setup
- Frontend integration

Week 1: Setup, Git, Spring Boot, PostgreSQL Basics

Goal

Validate that students can set up and run a backend project independently.

Topics

- Git workflow
- Spring Boot project setup
- PostgreSQL setup
- Basic REST API structure
- Postman usage
- Docker basics

Practical Task

Create a basic Spring Boot service.

Example APIs:

- GET /health
- GET /api/info

Expected Output

- Spring Boot app running locally
- Code pushed to Git
- Basic README added
- Postman request working
- PostgreSQL connected

Evaluation

- Can set up project independently
- Can use Git properly
- Can explain API request / response
- Can connect backend to PostgreSQL

Week 2: Banking CRUD API

Goal

Build backend APIs using a simple banking domain.

Topics

- REST API design
- Controller, Service, Repository layers
- DTO and Entity usage
- Hibernate / JPA entity mapping
- Request validation
- Exception handling
- PostgreSQL tables
- API testing with Postman

Practical Task

Build Banking CRUD APIs.

Example entities:

- Customer
- Bank Account
- Transaction
- Beneficiary

Example APIs:

- POST /api/customers
- GET /api/customers
- GET /api/customers/{id}
- POST /api/accounts
- GET /api/accounts
- GET /api/accounts/{accountId}
- POST /api/accounts/{accountId}/transactions
- GET /api/accounts/{accountId}/transactions
- POST /api/beneficiaries
- GET /api/beneficiaries
- DELETE /api/beneficiaries/{id}

Expected Output

- Working CRUD backend
- PostgreSQL integration
- Hibernate / JPA persistence working
- Validation and error handling
- Postman collection
- Clean Git commits
- README with setup steps

Evaluation

- Code structure
- API design
- DB design
- Entity mapping and repository usage
- Error handling
- Postman testing
- Git discipline

Week 3: Frontend Integration

Goal

Create a simple frontend and integrate it with the backend APIs.

Topics

- JavaScript frontend setup
- API integration
- Forms and validation
- List / detail pages
- Error handling in UI
- Environment-based API URL configuration

Frontend can be done using React, Angular, or simple JavaScript depending on the student level.

Practical Task

Create frontend screens for the banking APIs.

Example pages:

- Customer list
- Create customer
- Account list
- Account details
- Transaction history
- Beneficiary list
- Add beneficiary

Expected Output

- Frontend integrated with backend
- Create / list / view flows working
- API errors handled in UI
- Code pushed to Git
- Basic frontend README

Evaluation

- Can integrate APIs correctly
- Can debug frontend / backend issues
- Can explain request / response flow
- Can handle validation and error cases

Week 4: Keycloak Authentication and API Protection

Goal

Introduce authentication, authorization, JWT, and protected APIs.

Topics

- Authentication vs Authorization
- JWT basics
- OAuth2 basics
- Keycloak realm setup
- Keycloak client setup
- Users and roles
- Spring Boot JWT validation
- Role-based API protection
- 401 vs 403 debugging

Practical Task

Protect the banking APIs using Keycloak.

Example rules:

- GET /api/accounts -> requires valid JWT
- GET /api/accounts/{accountId} -> requires valid JWT
- POST /api/accounts -> requires admin role
- POST /api/transactions -> requires maker role
- DELETE /api/beneficiaries/{id} -> requires admin / checker role

Expected Output

- Keycloak realm configured
- Users and roles created
- Backend validates JWT
- Role-based access working
- Postman token testing completed

Evaluation

- Can explain JWT flow
- Can configure Keycloak client
- Can protect APIs
- Can debug 401 / 403 issues
- Can test APIs with bearer token

Week 5: Nginx Gateway and Docker Compose

Goal

Expose the application through Nginx Gateway and run the full stack using Docker Compose.

Topics

- Nginx reverse proxy
- Nginx as API Gateway
- Routing frontend / backend / Keycloak
- Gateway headers
- Basic access / error logs
- Docker Compose
- Full-stack local setup

Practical Task

Run the full system through Nginx as the single entry point.

Example routing:

- http://localhost:8080/ -> Frontend
- http://localhost:8080/api/customers -> Backend API through Nginx
- http://localhost:8080/api/accounts -> Backend API through Nginx
- http://localhost:8080/auth -> Keycloak

Expected Output

- Frontend routed through Nginx
- Backend APIs routed through Nginx
- Keycloak accessible through Nginx
- Docker Compose runs frontend + backend + DB + Keycloak + Nginx
- Basic Nginx logs enabled
- README updated

Evaluation

- Can explain request flow through Gateway
- Can configure Nginx reverse proxy
- Can read Nginx logs
- Can run full stack using Docker Compose

- Can debug routing issues

Optional Week 6: Final Capstone Review and Selection

Goal

Evaluate students and shortlist strong candidates for Banfico.

Final Capstone

Students should complete and demo:

Mini Banking / Open Banking Consent Management System

Expected Features

- Login using Keycloak
- Create customer
- Create account
- View account details
- View transaction history
- Add beneficiary
- Create consent request
- Approve / reject consent
- Protected backend APIs
- Role-based access
- Nginx Gateway
- PostgreSQL persistence
- Docker Compose setup
- Postman collection
- README documentation

Final Evaluation Criteria

- Code quality
- Problem-solving ability
- Debugging skills
- Git discipline
- API design
- Security understanding
- Frontend / backend integration
- Spring Boot and Hibernate / JPA understanding
- Keycloak understanding
- Nginx Gateway understanding
- Docker setup
- Ownership and communication
- Basic banking / open banking understanding

Banfico Team Involvement

Banfico team involvement should be limited to practical review and guidance.

- Weekly technical sessions
- Session 1 task walkthrough limited to 1.5 hours
- Code review
- Architecture guidance
- Debugging support
- Final evaluation
- Best practices from real projects

Banfico team should avoid spending time on basic Java, JavaScript, SQL, Git, Spring Boot, or Hibernate training. Candidates should complete these topics through self-learning before or alongside the program.

Selection Approach

Students can be filtered progressively.

After Week 1: Filter students who cannot set up / run projects independently.

After Week 2: Shortlist students who can build clean backend APIs using Spring Boot, PostgreSQL, and Hibernate / JPA.

After Week 3: Shortlist students who can integrate frontend and backend.

After Week 4: Shortlist students who understand JWT, Keycloak, and protected APIs.

After Week 5 / 6: Select students who can complete the full-stack banking project with Nginx Gateway and Docker setup.

Final Outcome

By the end of 5-6 weeks, strong students should be able to:

- Build Java Spring Boot APIs
- Use Hibernate / JPA for persistence
- Work with PostgreSQL
- Build simple frontend integrations
- Use Git properly
- Run services using Docker
- Configure Nginx as API Gateway
- Configure Keycloak
- Protect APIs using JWT
- Debug practical full-stack issues
- Understand basic banking / open banking application flow

This program will help Banfico identify the best students from the group and bring the strongest candidates into the team.